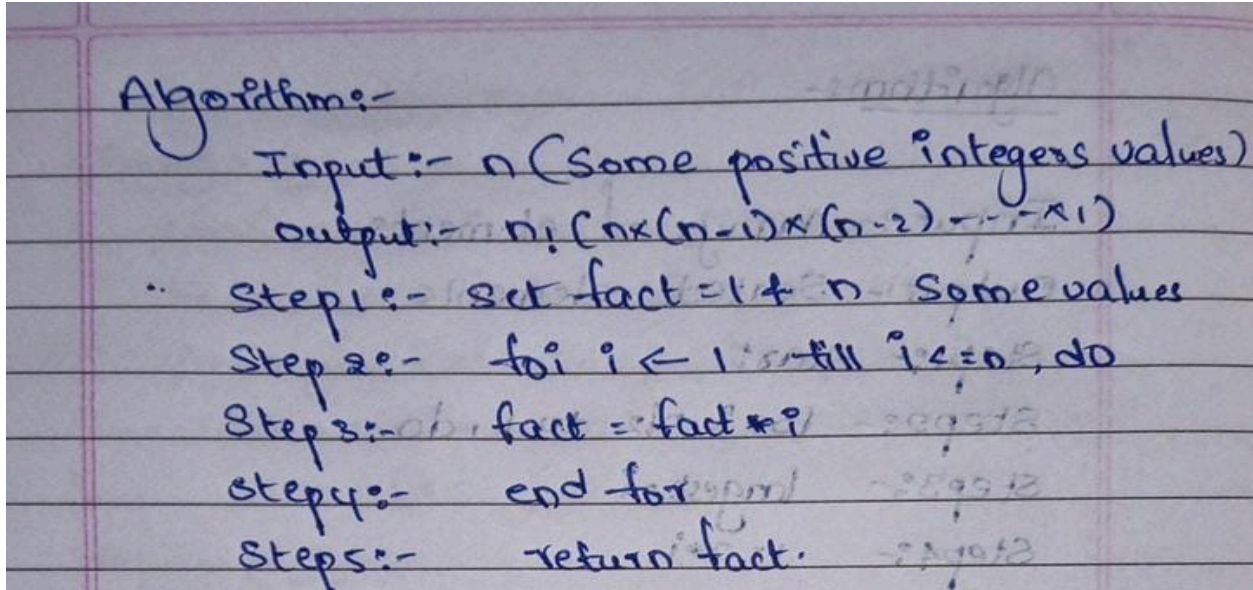


PRACTICAL-4

AIM: Implementation and Time analysis of factorial program using iterative and recursive method.

Factorial Using Iterative Method

ALGORITHM:



Algorithm:-
Input:- n (Some positive integers values)
Output:- $n!$ ($n \times (n-1) \times (n-2) \times \dots \times 1$)
Step 1:- Set fact = 1 & n Some values
Step 2:- for $i \leftarrow 1$ till $i \leq n$, do
Step 3:- fact = fact * i
Step 4:- end for
Step 5:- return fact.

CODE:

```
#include <iostream>
using namespace std;
```

```
class Factorial
{
public:
    int fact(int n)
    {
        int result = 1;

        for (int i = 1; i <= n; i++)
        {
            result = result * i;
        }
    }
}
```



```
        return result;
    }
};

int main()
{
    int n,result;
    Factorial obj;

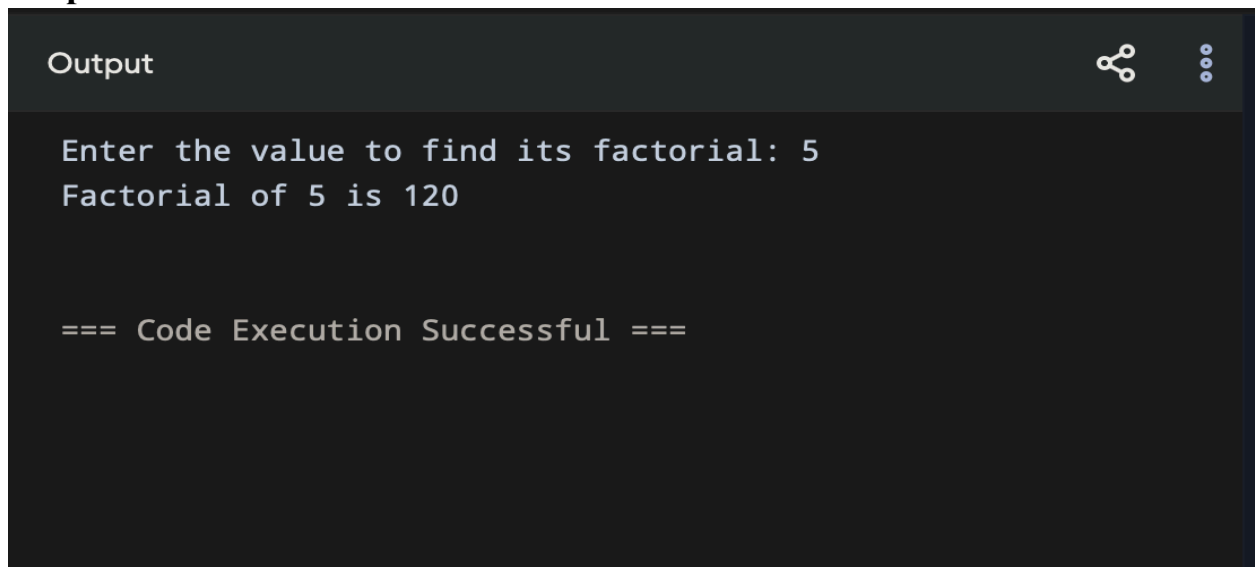
    cout << "Enter the value to find its factorial: ";
    cin >> n;

    result = obj.fact(n);

    cout << "Factorial of " << n << " is " << result << endl;

    return 0;
}
```

Output:



```
Output

Enter the value to find its factorial: 5
Factorial of 5 is 120

=== Code Execution Successful ===
```

Time Complexity:

Time Complexity:-

```

for (int i = 1; i <= n; i++) → n
{
    result = result * i; → 1
}
    
```

$T(n) = n \times 1 \Rightarrow T(n) = O(n)$

Best case:-

```

for (int i = 1; i <= n; i++) → n
{
    result = result * i; → 1
}
    
```

$T(n) = n \times 1$
 $T(n) = O(n)$

Average Case:-

```

for (int i = 1; i <= n; i++) → n
{
    result = result * i; → 1
}
    
```

$T(n) = n \times 1$
 $T(n) = O(n)$

Worst Case:-

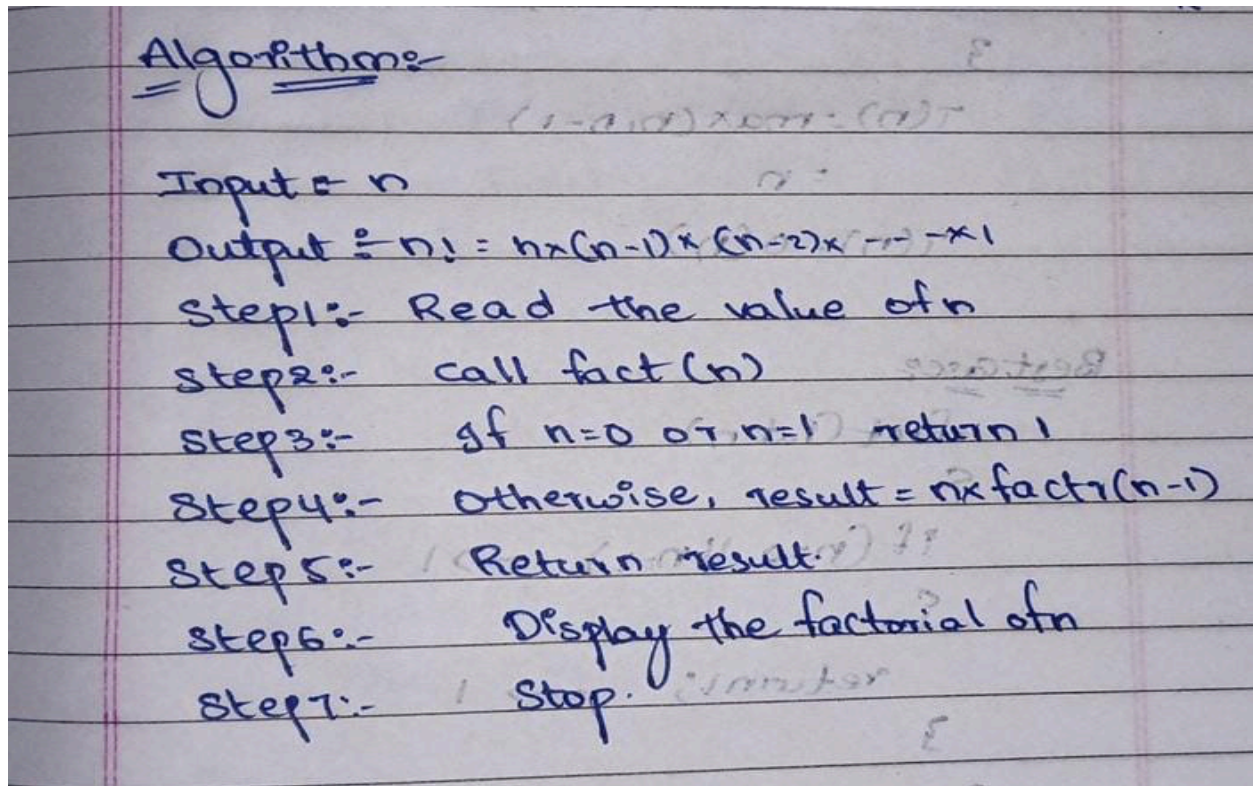
```

for (int i = 1; i <= n; i++) → n
{
    result = result * i → 1
}
    
```

$T(n) = n \times 1$
 $T(n) = O(n)$

Factorial Using Recursive Method:

Algorithm



Code:

```
#include <iostream>

using namespace std;

class Factorial

{

public:

    int factr(int n)
```



```
{  
  
    if (n == 0 || n == 1) {  
  
        return 1;  
  
    }  
  
    else  
  
    {  
  
        int result = n * factr(n -  
1);  
  
        return result;  
  
    }  
  
};  
  
int main()  
  
{  
  
    int n, result;  
  
    // Create an object of the class  
  
    Factorial obj;  
  
    cout << "Enter the value to  
find its factorial: ";
```



```
cin >> n;  
  
result = obj.factr(n);  
  
cout << "Factorial of " << n  
<< " is " << result << endl;  
  
return 0;  
  
}
```

Output:

Output

```
Enter the value to find its factorial: 6  
Factorial of 6 is 720
```

```
=== Code Execution Successful ===
```

TIME COMPLEXITY:

Time Complexity.

int fact(int n)

{

if (n=0 || n=1) → 1

return 1;

else

result = n * fact(n-1) → n-1

return result;

}

$T(n) = \max(n, n-1)$

= n

$T(n) = O(n)$

Best case:

int fact(int n)

{

if (n=0 || n=1) → 1

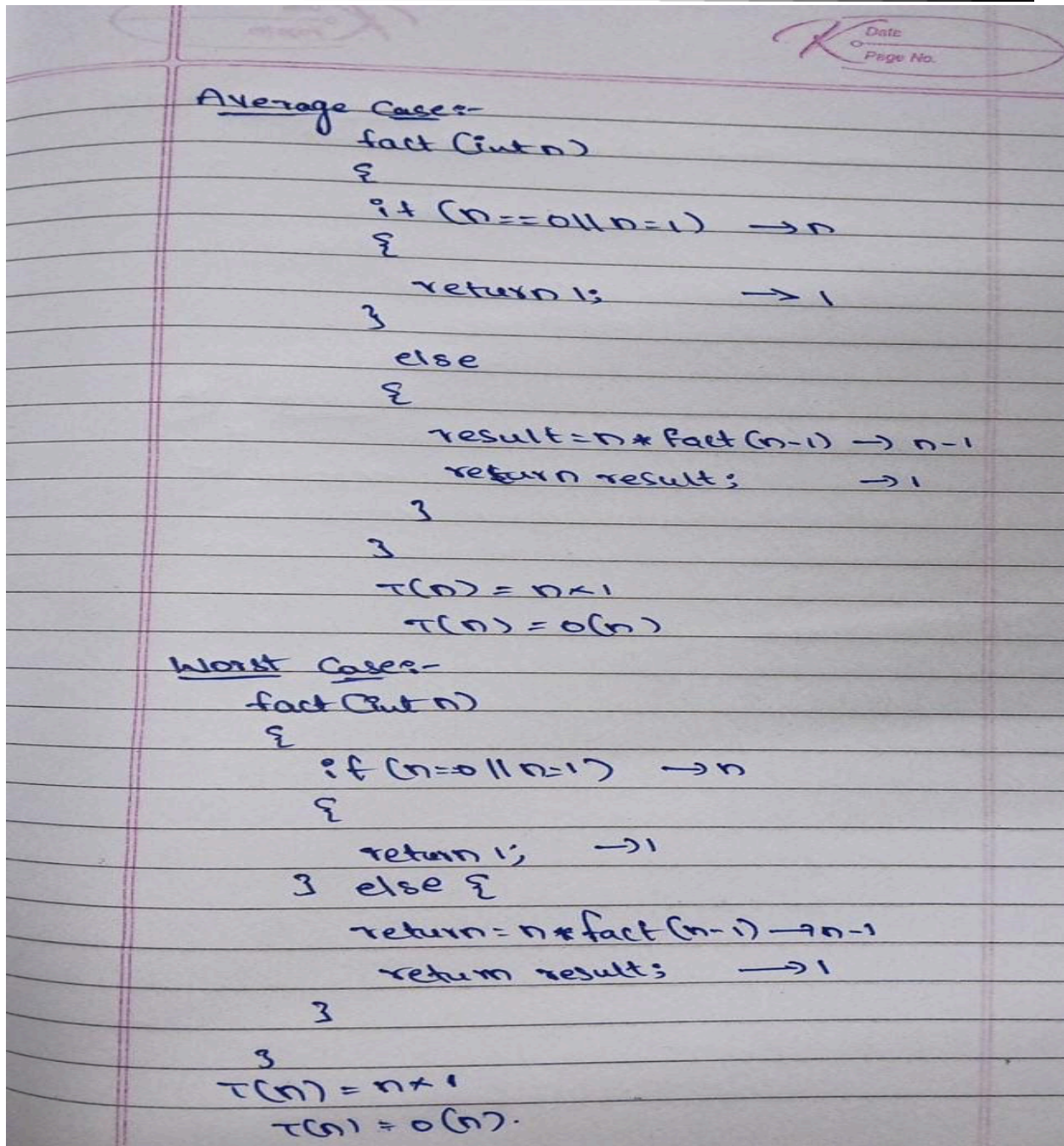
return 1;

}

}

$T(n) = 1 \times 1$

$T(n) = O(1)$



Conclusion:

The factorial program was successfully implemented using both iterative and recursive methods. Both methods produce the same result, but the recursive method uses function calls and stack space. The time complexity of both methods is $O(n)$, making them efficient for calculating factorials.